

Mount Kenya University

Unlocking Infinite Possibilities

NAME: PETER SHEMA YA

ADM NUMBER: BSCCS/2024/56268

COURSE: COMPUTER SCIENCE

UNIT CODE: BIT3208

UNIT NAME: ADVANCED WEB DESIGN AND DEVELOPMENT

LECTURER NAME: MICHAEL NYORO

SEMESTER: 3.2

PROJECT TITLE: WEEK 6 & 7

SELECTED TECHNOLOGIES: PHP + MySQL

GITHUB LINK: <https://github.com/Coutts007/Advancedweb>

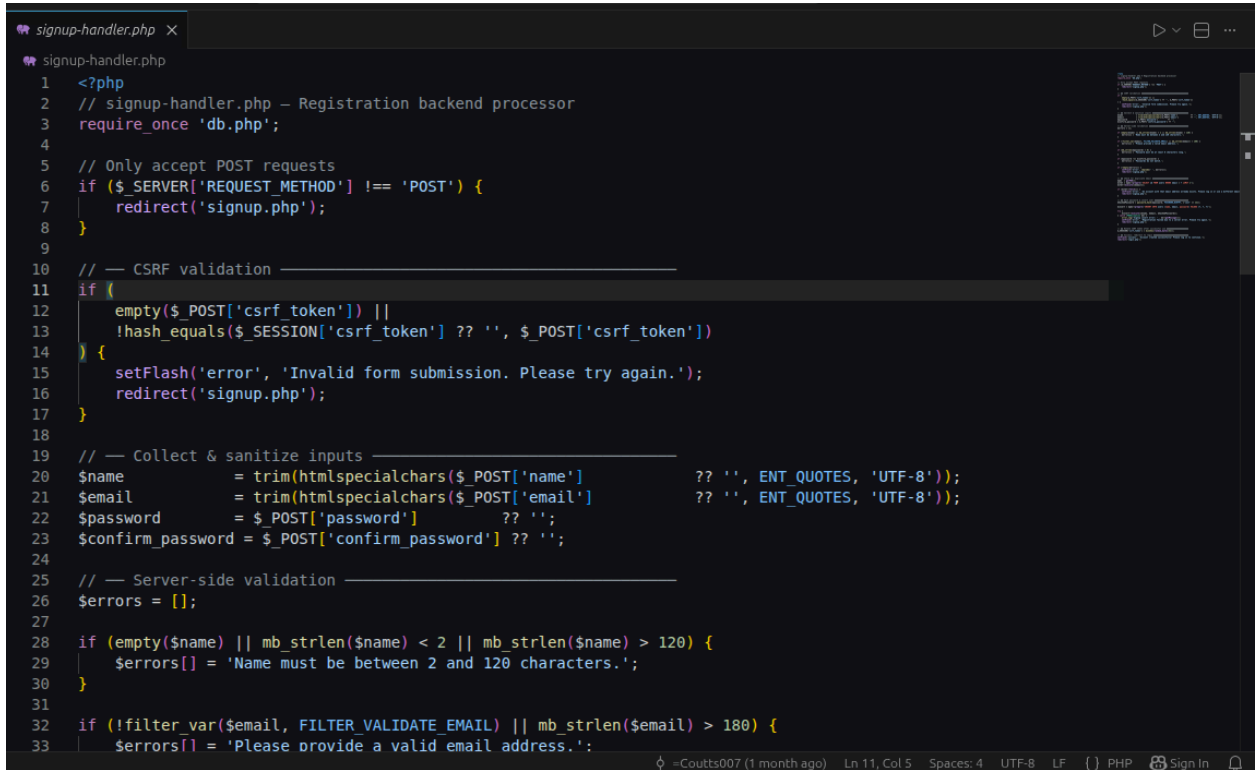
TABLE OF CONTENT

WEEK 6: DATABASE INTEGRATION AND CRUD OPERATIONS.....	3
1. Database Integration in Web Applications.....	3
Fig. 1.1: Code showing complete system-database integration and connection.....	3
2. CRUD Operations.....	4
Fig. 2.1: CRUD operations performance through the system's admin dashboard.....	4
3. Connect a Web Application to a Database.....	4
Fig. 3.1: MySQL query to create database and respective tables for the system.....	4
Fig. 3.2: Complete database creation and connection with users successfully updated in the database.....	5
4. Create, Read, Update, and Delete Records Using PHP and MySQL.....	5
Fig. 4.1: Created new tables to the database using MySQL.....	5
Fig. 4.2: Selected and read the user table using MySQL.....	6
5. User-Friendly Forms for Data Entry.....	6
Fig. 5.1: Implemented an efficient and user friendly signup and login forms for data entry.	6
6. Test and Debug Database-Driven Applications.....	7
Fig. 6.1: A successful user login implying a successful database connection.....	7
WEEK 7: USER AUTHENTICATION AND SESSION MANAGEMENT.....	8
1. Create User Registration and Login Forms.....	8
Fig. 1.1: A functional user registration form.....	8
Fig. 1.2: A functional user login form.....	8
2. Password Hashing for Security.....	9
Fig. 2.1: Password secured using hashing in database storage.....	9
3. User Sessions Using PHP.....	9
Fig. 3.1: PHP code for managing user sessions successfully.....	9
4. Restrict Access to Authorized Users Only.....	10
Fig. 4.1: Admin dashboard only accessed by the system admin.....	10
Fig. 4.2: The system manager is granted authority and access to the manager dashboard.....	10
Fig. 4.3: The client given authority to only the client dashboard.....	11
5. Secure Logout Functionality.....	11
Fig. 5.1: A successfully implemented secure system logout.....	11
6. Basic Security Practices in Web Applications.....	12
Fig. 6.1: Applied basic security practice to ensure strong password input by users.....	12

WEEK 6: DATABASE INTEGRATION AND CRUD OPERATIONS

1. Database Integration in Web Applications

Fig. 1.1: Code showing complete system-database integration and connection.

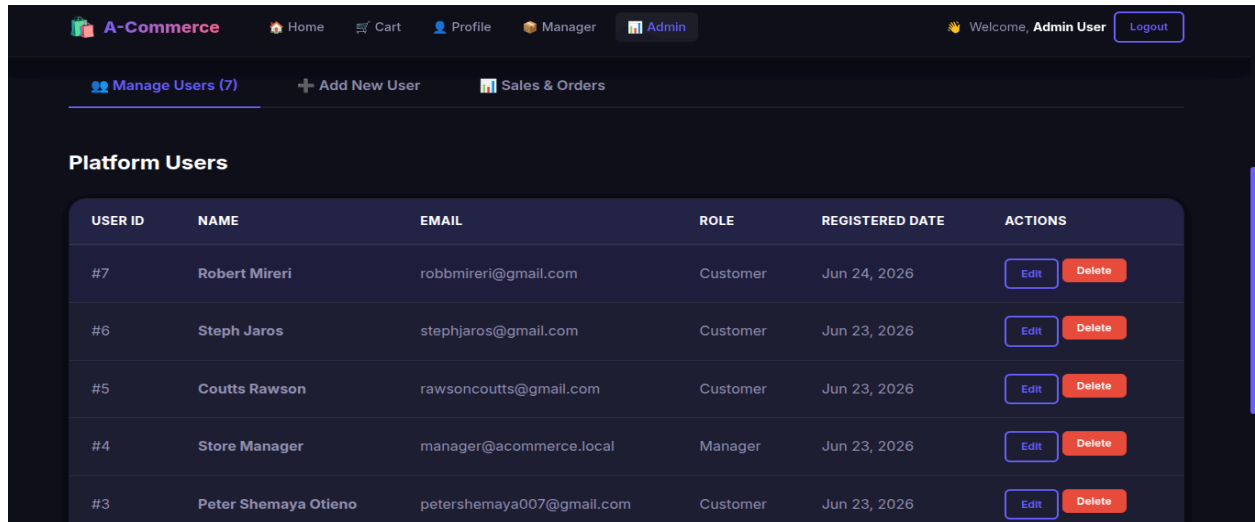
A screenshot of a code editor with a dark theme. The editor shows a file named 'signup-handler.php'. The code is PHP and includes comments, variable assignments, conditional logic for CSRF validation and input sanitization, and error handling. The status bar at the bottom indicates the file is at line 11, column 5, with 4 spaces, UTF-8 encoding, LF line endings, and is a PHP file. There is also a 'Sign In' button in the bottom right corner of the editor interface.

```
1 <?php
2 // signup-handler.php - Registration backend processor
3 require_once 'db.php';
4
5 // Only accept POST requests
6 if ($SERVER['REQUEST_METHOD'] !== 'POST') {
7     redirect('signup.php');
8 }
9
10 // --- CSRF validation ---
11 if (
12     empty($_POST['csrf_token']) ||
13     !hash_equals($_SESSION['csrf_token'] ?? '', $_POST['csrf_token'])
14 ) {
15     setFlash('error', 'Invalid form submission. Please try again.');
```

Explanation: This script establishes a secure database gateway connection, utilizing conditional request routing, input token validation arrays, and server-side cleansing protocols before handling state requests.

2. CRUD Operations

Fig. 2.1: CRUD operations performance through the system's admin dashboard.

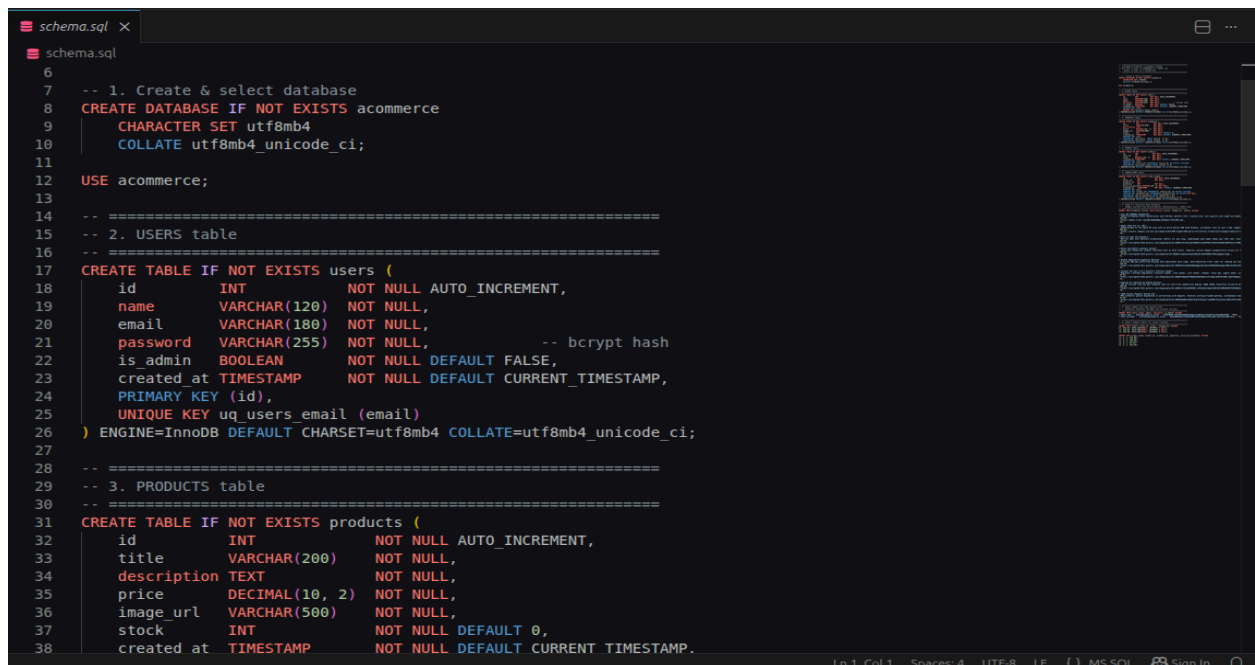


USER ID	NAME	EMAIL	ROLE	REGISTERED DATE	ACTIONS
#7	Robert Mireri	robbmireri@gmail.com	Customer	Jun 24, 2026	<button>Edit</button> <button>Delete</button>
#6	Steph Jaros	stephjaros@gmail.com	Customer	Jun 23, 2026	<button>Edit</button> <button>Delete</button>
#5	Coutts Rawson	rawsoncoutts@gmail.com	Customer	Jun 23, 2026	<button>Edit</button> <button>Delete</button>
#4	Store Manager	manager@acommerce.local	Manager	Jun 23, 2026	<button>Edit</button> <button>Delete</button>
#3	Peter Shemaya Otieno	petershemaya007@gmail.com	Customer	Jun 23, 2026	<button>Edit</button> <button>Delete</button>

Explanation: This user console renders a tabular interface mapping unique identifier keys against stored user profiles, exposing interactive controls to safely execute backend modifications.

3. Connect a Web Application to a Database

Fig. 3.1: MySQL query to create database and respective tables for the system.



```
6
7 -- 1. Create & select database
8 CREATE DATABASE IF NOT EXISTS acommerce
9   CHARACTER SET utf8mb4
10  COLLATE utf8mb4_unicode_ci;
11
12 USE acommerce;
13
14 -- =====
15 -- 2. USERS table
16 -- =====
17 CREATE TABLE IF NOT EXISTS users (
18   id INT NOT NULL AUTO_INCREMENT,
19   name VARCHAR(120) NOT NULL,
20   email VARCHAR(180) NOT NULL,
21   password VARCHAR(255) NOT NULL, -- bcrypt hash
22   is_admin BOOLEAN NOT NULL DEFAULT FALSE,
23   created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
24   PRIMARY KEY (id),
25   UNIQUE KEY uq_users_email (email)
26 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
27
28 -- =====
29 -- 3. PRODUCTS table
30 -- =====
31 CREATE TABLE IF NOT EXISTS products (
32   id INT NOT NULL AUTO_INCREMENT,
33   title VARCHAR(200) NOT NULL,
34   description TEXT NOT NULL,
35   price DECIMAL(10, 2) NOT NULL,
36   image_url VARCHAR(500) NOT NULL,
37   stock INT NOT NULL DEFAULT 0,
38   created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP.
```

Explanation: This structural query script defines the table blueprint, instantiating target primary keys, attribute data constraints, and explicit column constraints.

Fig. 3.2: Complete database creation and connection with users successfully updated in the database

The screenshot shows the phpMyAdmin interface with the 'acommerce' database selected. The 'users' table is displayed with the following data:

id	name	email	password	is_admin	created_at	role
1	Admin User	admin@acommerce.local	\$2y\$10\$OSZH8BvzRqW00vgURFbjOjrgotNXNoiPLfLqmAVEfa...	1	2026-06-23 13:24:37	adm
2	John Deere	customer@acommerce.local	\$2y\$10\$NJKw1ffUGSAcZWR/H3oRcudREnioPMsLZOS.GjSxwly...	0	2026-06-23 13:24:37	cust
3	Peter Shemaya	petershemaya007@gmail.com	\$2y\$12\$V7VzGU4bc828cvWCC1N3eAXoV5Yk24n9Qftc9bQoVg...	0	2026-06-23 13:25:28	cust
4	Otieno Store Manager	manager@acommerce.local	\$2y\$12\$6eiHTrL8vpF9XN.28ig2nu8mq7GC7iORJlpvZR0D/7P...	0	2026-06-23 14:40:29	mar
5	Coutts Rawson	rawsoncoutts@gmail.com	\$2y\$12\$QLtLsUuV4WmKXqGa7Ros9eRzBNiOaOBmPkpTKs15m4F...	0	2026-06-23 15:08:36	cust
6	Steph Jaros	stephjaros@gmail.com	\$2y\$12\$SoEtel7VmN6d/S4UfzA3YJO/Ws7MeMaBTLGJn755mgk3...	0	2026-06-23 17:10:01	cust
7	Robert Mireri	robbmireri@gmail.com	\$2y\$12\$7arGcGalwucPwX5ge6fJ.XZyPhi82iRadI2ib8FOG...	0	2026-06-24 10:46:51	cust

Explanation: This administrative backend view validates a live connection pipeline, showing populated relational rows safely committed to the central storage layout.

4. Create, Read, Update, and Delete Records Using PHP and MySQL

Fig. 4.1: Created new tables to the database using MySQL.

The screenshot shows the 'Run SQL query/queries on table acommerce.users:' window in phpMyAdmin. The SQL query is as follows:

```

1 CREATE TABLE IF NOT EXISTS users (
2   id      INT          NOT NULL AUTO_INCREMENT,
3   name    VARCHAR(120) NOT NULL,
4   email   VARCHAR(180) NOT NULL,
5   password VARCHAR(255) NOT NULL,           -- bcrypt hash
6   is_admin BOOLEAN     NOT NULL DEFAULT FALSE,
7   created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
8   PRIMARY KEY (id),
9   UNIQUE KEY uq_users_email (email)
10 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

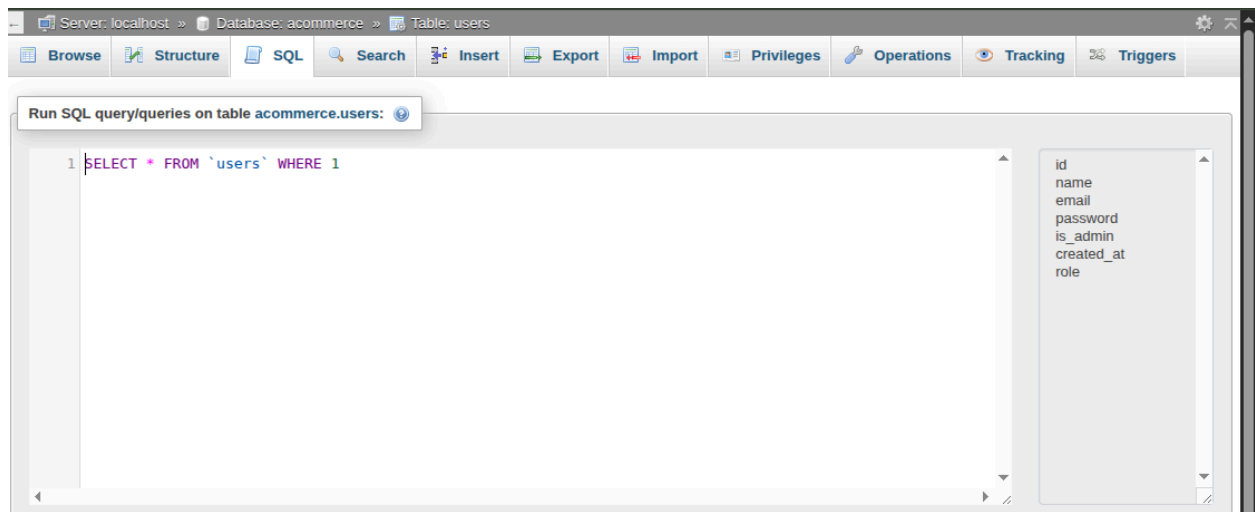
```

On the right side of the query editor, a preview of the table structure is shown:

id	name	email	password	is_admin	created_at	role
----	------	-------	----------	----------	------------	------

Explanation: This view displays the schema structure inside the administration control layer, verifying that structural constraints and database components are correctly configured.

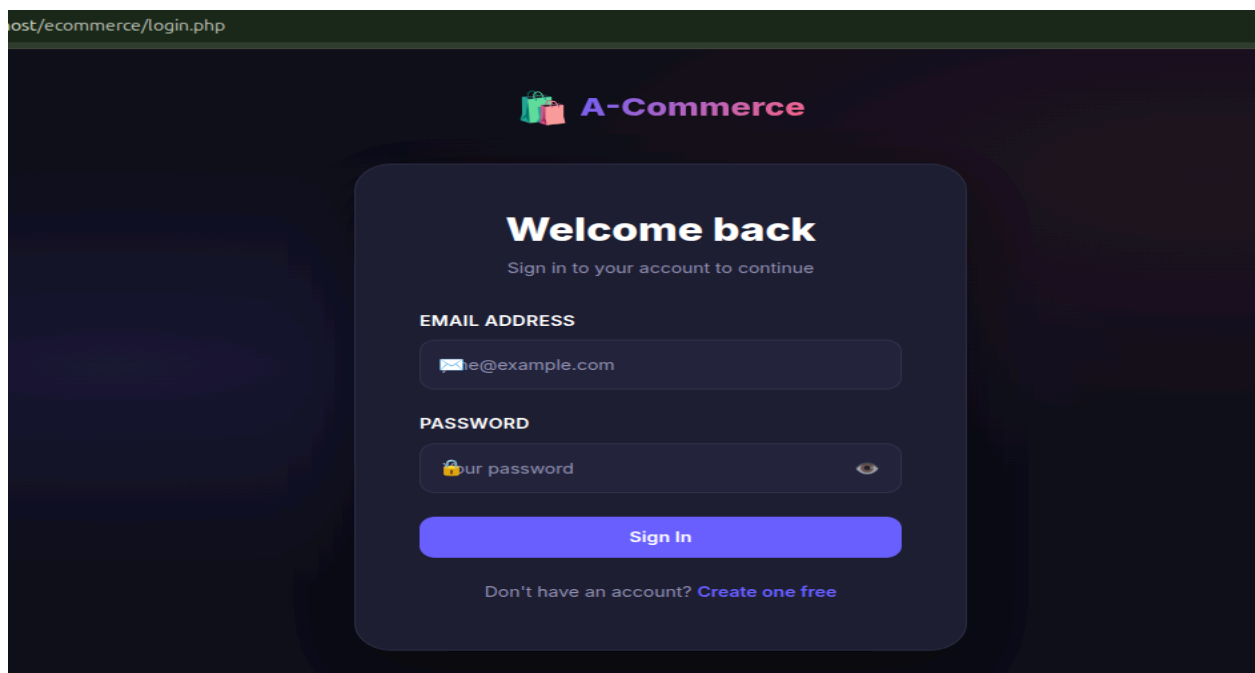
Fig. 4.2: Selected and read the user table using MySQL.



Explanation: This interface displays active table records returned via dynamic selection streams, mapping row tuples accurately across predefined data columns.

5. User-Friendly Forms for Data Entry

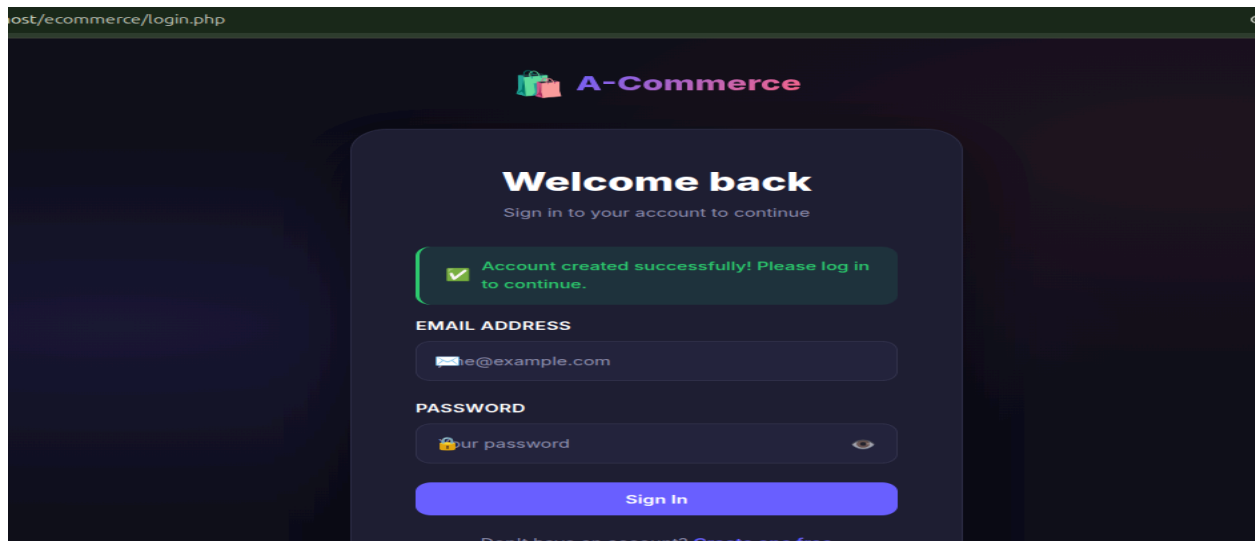
Fig. 5.1: Implemented an efficient and user friendly signup and login forms for data entry.



Explanation: This module presents standard identity inputs with responsive field hints, enforcing user validation criteria before passing text buffers upstream.

6. Test and Debug Database-Driven Applications

Fig. 6.1: A successful user login implying a successful database connection.

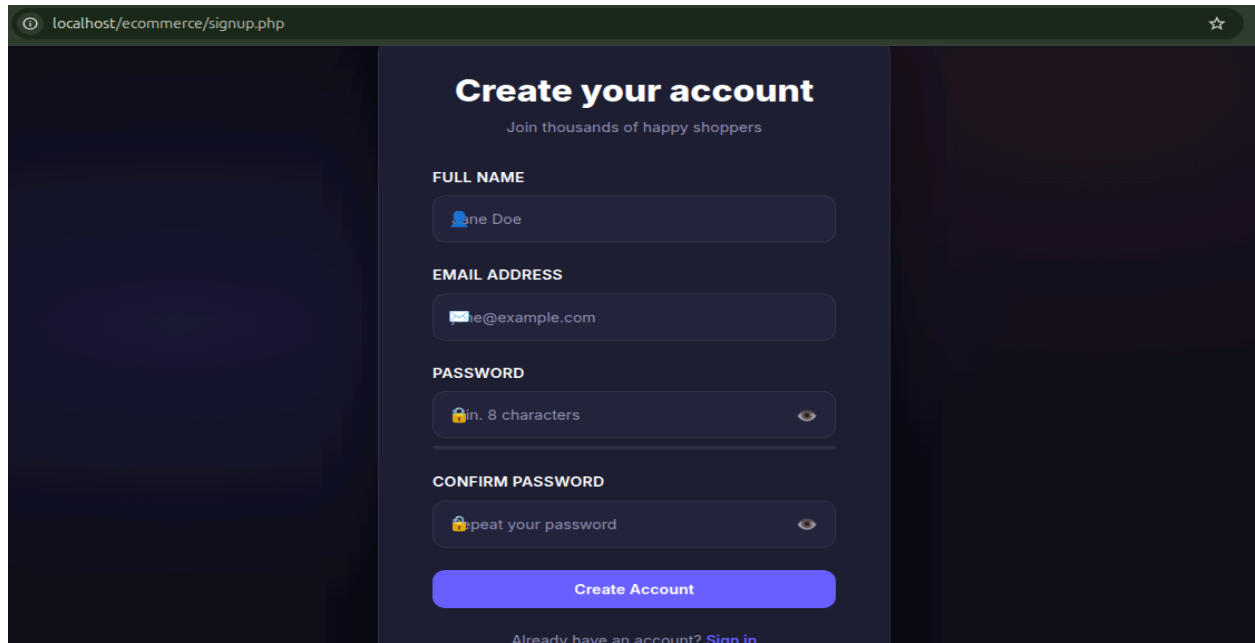


Explanation: This dashboard capture confirms a valid login pipeline execution, indicating successful connection processing and session token allocation.

WEEK 7: USER AUTHENTICATION AND SESSION MANAGEMENT

1. Create User Registration and Login Forms

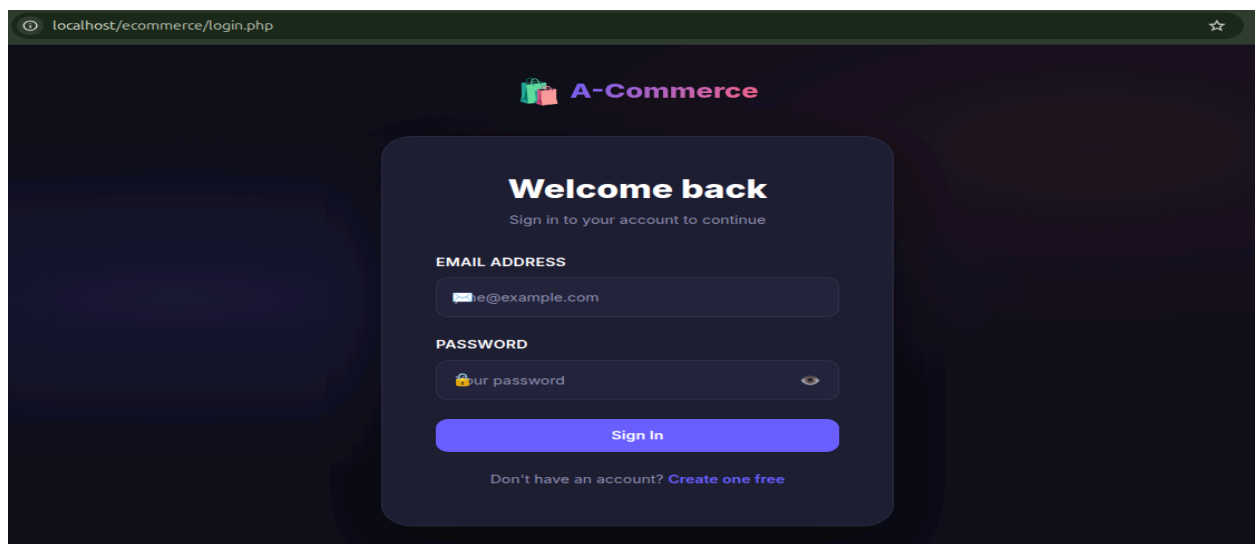
Fig. 1.1: A functional user registration form.



The screenshot shows a web browser window with the address bar displaying 'localhost/ecommerce/signup.php'. The main content area features a dark-themed registration form titled 'Create your account' with the subtitle 'Join thousands of happy shoppers'. The form includes four input fields: 'FULL NAME' (containing 'John Doe'), 'EMAIL ADDRESS' (containing 'john@example.com'), 'PASSWORD' (containing '12345678' with a strength indicator 'min. 8 characters' and a toggle icon), and 'CONFIRM PASSWORD' (containing 'Repeat your password' with a toggle icon). A blue 'Create Account' button is positioned below the fields. At the bottom, a link reads 'Already have an account? [Sign in](#)'.

Explanation: This interface manages account initialization parameters, checking input constraints on email strings and profile variables before execution.

Fig. 1.2: A functional user login form.

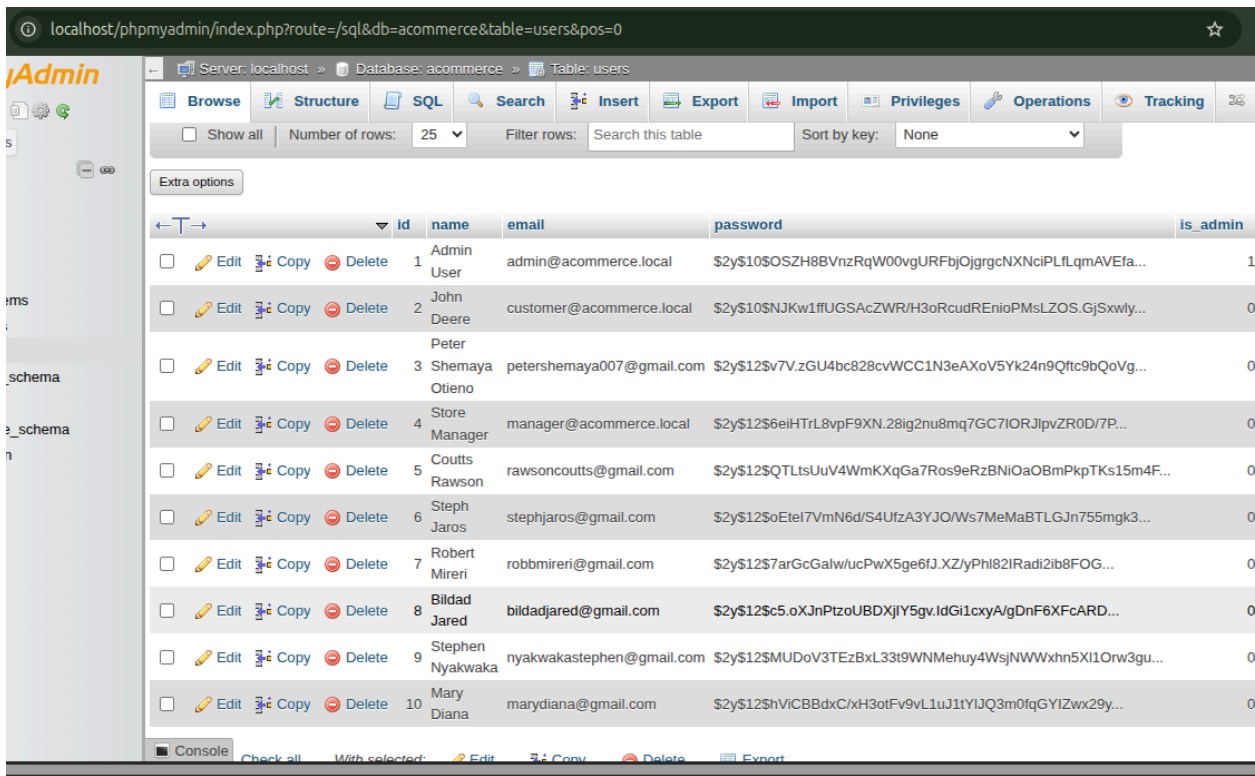


The screenshot shows a web browser window with the address bar displaying 'localhost/ecommerce/login.php'. The main content area features a dark-themed login form titled 'Welcome back' with the subtitle 'Sign in to your account to continue'. The form includes two input fields: 'EMAIL ADDRESS' (containing 'john@example.com') and 'PASSWORD' (containing '12345678' with a toggle icon). A blue 'Sign In' button is positioned below the fields. At the bottom, a link reads 'Don't have an account? [Create one free](#)'.

Explanation: This specialized gate renders masked password fields and account keys to protect user login access details during transmission.

2. Password Hashing for Security

Fig. 2.1: Password secured using hashing in database storage.



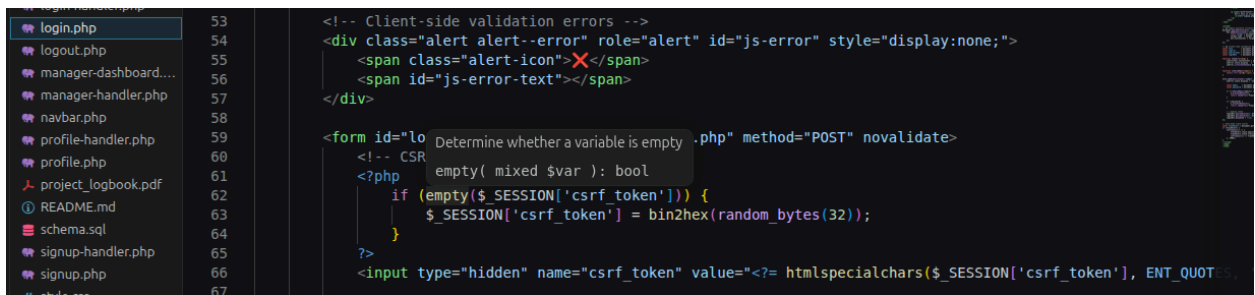
The screenshot shows the phpMyAdmin interface for a database named 'acommerce'. The 'users' table is selected, showing 25 rows. The table has columns: id, name, email, password, and is_admin. The 'password' column contains long, unique, encrypted cryptographic hash strings. The interface includes a top navigation bar with options like Browse, Structure, SQL, Search, Insert, Export, Import, Privileges, Operations, and Tracking. A left sidebar shows the database structure. The main area displays the table data with options to edit, copy, or delete each row.

	id	name	email	password	is_admin
☐	1	Admin User	admin@acommerce.local	\$2y\$10\$OSZH8BvnzRqW00vgURFbJOjgrgcNXNciPLILqmAVEfa...	1
☐	2	John Deere	customer@acommerce.local	\$2y\$10\$NJKw1ffUGSACZWR/H3oRcudREnioPMsLZOS.GjSxwly...	0
☐	3	Peter Shemaya	petershemaya007@gmail.com	\$2y\$12\$5v7V.zGU4bc828cvWCC1N3eAXoV5Yk24n9Qftc9bQoVg...	0
☐	4	Otieno Store Manager	manager@acommerce.local	\$2y\$12\$6eiHTrL8vpF9XN.28ig2nu8mq7GC7IORJlJpvZR0D/7P...	0
☐	5	Coutts Rawson	rawsoncoutts@gmail.com	\$2y\$12\$QTLtsUuV4WmKXqGa7Ros9eRzBNiOaOBmPkpTKs15m4F...	0
☐	6	Steph Jaros	stephjaros@gmail.com	\$2y\$12\$SoEtel7V/mN6d/S4UfzA3YJO/Ws7MeMaBTLGJn755mgk3...	0
☐	7	Robert Mireri	robbmireri@gmail.com	\$2y\$12\$7arGcGalwucPwX5ge6fJ.XZJyPhl82lRadi2ib8FOG...	0
☐	8	Bildad Jared	bildadjared@gmail.com	\$2y\$12\$c5.oXJnPtzoUBDXjlY5gv.IdGi1cxyA/gDnF6XFcARD...	0
☐	9	Stephen Nyakwaka	nyakwakastephen@gmail.com	\$2y\$12\$MUDoV3TEzBxL33t9WNMehuy4WsjNWWxhn5Xl1Orw3gu...	0
☐	10	Mary Diana	marydiana@gmail.com	\$2y\$12\$ShViCBBdxC/xH3otFv9vL1uJ1tYUQ3m0fGqYIZwx29y...	0

Explanation: This backend table partition highlights sensitive credentials stored strictly as unique, encrypted cryptographic hash strings.

3. User Sessions Using PHP

Fig. 3.1: PHP code for managing user sessions successfully.



The screenshot shows a code editor with PHP code for managing user sessions. The code includes client-side validation errors, a form for login, and a CSRF token generation and validation process. The code is structured as follows:

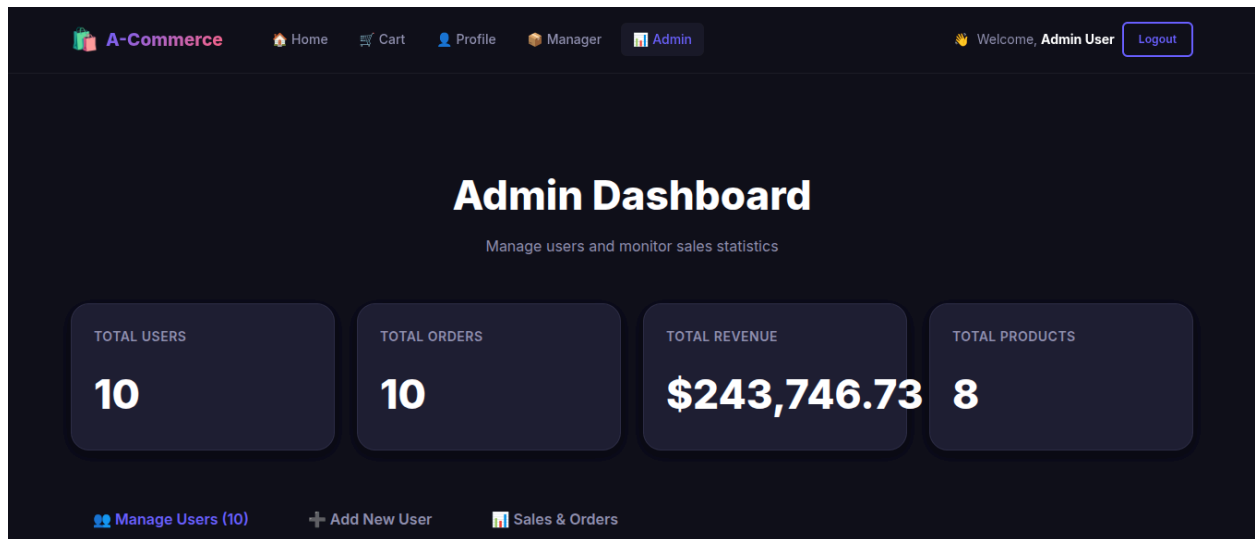
```
<!-- Client-side validation errors -->
<div class="alert alert--error" role="alert" id="js-error" style="display:none;">
  <span class="alert-icon">X</span>
  <span id="js-error-text"></span>
</div>

<form id="login" method="POST" novalidate>
  <!-- CSR -->
  <?php
    empty( mixed $var ): bool
    if (empty($SESSION['csrf_token'])) {
      $SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
  ?>
  <input type="hidden" name="csrf_token" value="<?php echo htmlspecialchars($SESSION['csrf_token'], ENT_QUOTES);"/>
```

Explanation: This structural block initializes the global state tracking array, verifying session tokens before rendering protected dashboard components.

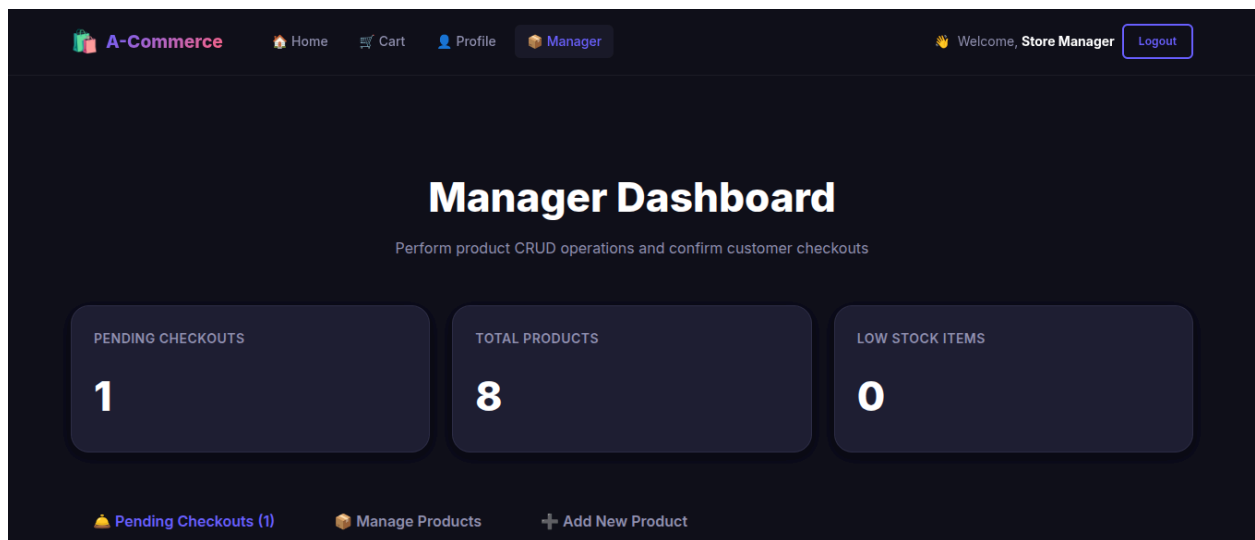
4. Restrict Access to Authorized Users Only

Fig. 4.1: Admin dashboard only accessed by the system admin.



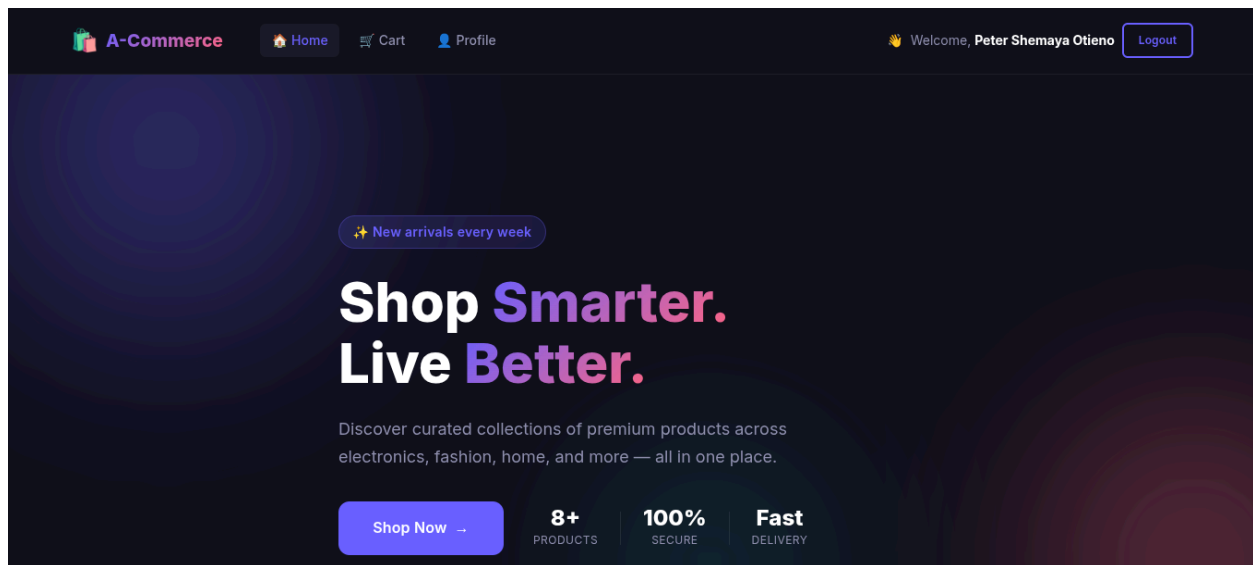
Explanation: This control panel checks for specific admin session privileges, rendering high-level management tools exclusive to that role.

Fig. 4.2: The system manager is granted authority and access to the manager dashboard



Explanation: This workspace workspace isolates system configuration views, restricting entry tokens to verified managerial profiles.

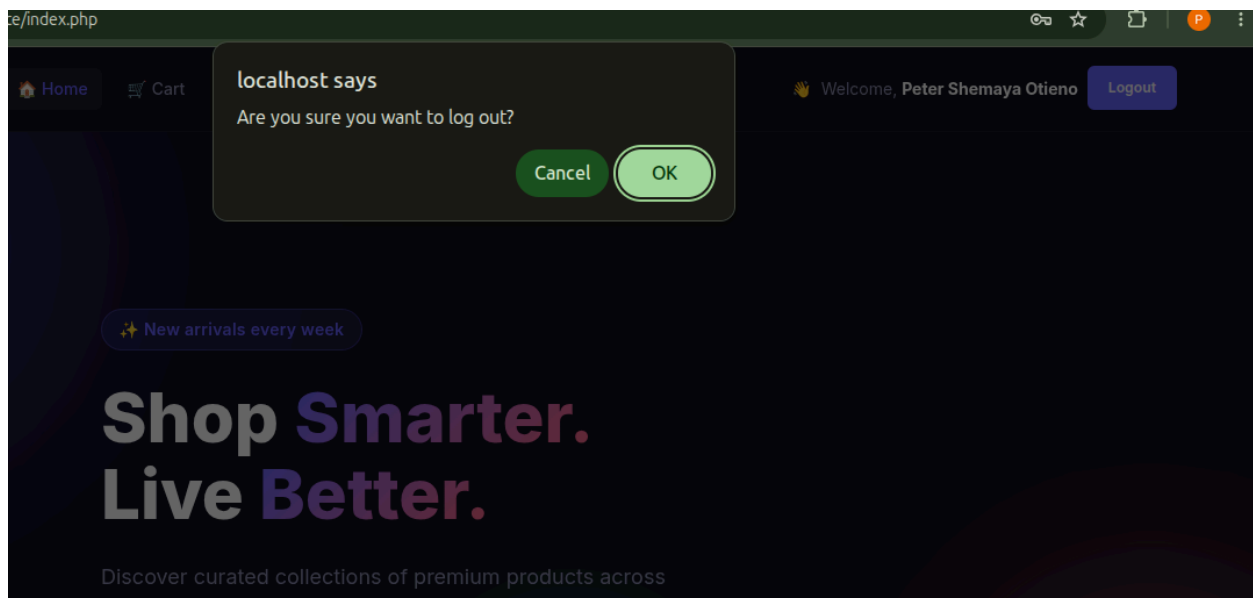
Fig. 4.3: The client given authority to only the client dashboard.



Explanation: This interface presents a single, protected dashboard partition restricted strictly to client profile elements using verified session variables.

5. Secure Logout Functionality

Fig. 5.1: A successfully implemented secure system logout



Explanation: This screen confirms the complete destruction of active user tracking state variables, redirecting the browser session to the initial landing portal.

6. Basic Security Practices in Web Applications

Fig. 6.1: *Applied basic security practice to ensure strong password input by users.*

ce/signup.php

FULL NAME

eter Shemaya Otieno

EMAIL ADDRESS

tershemaya007@gmail.com

PASSWORD

em123



Very Weak

CONFIRM PASSWORD

.....



Create Account

Already have an account? [Sign in](#)

Explanation: This input configuration applies regular expression rules to require specific character counts, preventing trivial password setups at the browser level.